

Применение статистических методов в детекторах сетевых аномалий

Техническое задание для backend-разработки

Обоснование подхода и выбора методов

Почему статистика как дополнение, а не как основа. Проект работает в режиме офлайн-анализа рсар, где жёсткие правила детекции (порог rps, число уникальных dstIP, доля одного JA3) уже дают надёжный сигнал. Статистические методы здесь применяются для решения трёх задач, которые жёсткие пороги решают плохо: адаптация к разному «профилю» входных файлов (один рсар снят с пограничного маршрутизатора, другой — с офисной подсети), детекция плавных аномалий (постепенное наращивание нагрузки), и уменьшение ложных срабатываний на легитимных пиках (рассылка корпоративной почты, бэкап в окно). Статистика — адаптивный слой поверх правил, не замена им.

Особенность офлайн-сценария. В отличие от online-NDR, у нас нет накопленной истории трафика; baseline для статистик строится **из самого рсар-файла**. Это требует аккуратности: использовать «первые N% файла как эталон» — неверно, если атака начинается с первого пакета. Корректный подход — скользящие окна и относительные сравнения соседних окон, чтобы оценка нормы и оценка аномалии всегда были близки по времени.

Какие методы выбраны и почему.

- **Скользящее среднее (SMA) и EWMA** — сглаживание шума, выделение тренда; EWMA реагирует быстрее на новые изменения, SMA — стабильнее [1].
- **Стандартное отклонение и Z-score** — классическая мера «насколько текущее окно отличается от соседних»; работает на гауссовских данных, но в реальном трафике распределения тяжелохвостые [2], поэтому Z-score применяется только как один из сигналов, а не единственный.
- **Перцентили (p95, p99) и IQR** — робастные альтернативы среднему/ σ : устойчивы к выбросам, не требуют предположения о нормальности [3]. Особенно хороши для метрик с тяжёлым хвостом (длительность потоков, размер пакетов).

- **Медианное абсолютное отклонение (MAD)** — ещё более робастная замена σ , не «загрязняется» самой аномалией при её попадании в baseline [4].
- **Относительный рост между окнами** — простейший адаптивный сигнал, не требует исторического baseline; в офлайн-сценарии часто работает лучше Z-score.

Принцип комбинирования. Алерт детектора срабатывает по существующему жёсткому правилу. Статистические сигналы добавляются к нему как **поля контекста** (z-score, перцентиль, рост) и опционально как **мягкое снижение порога**: если за окно зафиксирована стат-аномалия, абсолютный порог детектора понижается на конфигурируемый коэффициент. Это позволяет ловить медленные атаки, не теряя устойчивости к шуму.

1 Расширение структуры окна статистики

Файл: internal/packet/stats.go

В структуру окна (или эквивалент в текущей реализации) добавить поля для исторических значений ключевых метрик:

```

1 type MetricHistory struct {
2     Values    []float64 // значения метрики за последние N окон
3     Capacity  int      // максимальная длина истории; например 30
4 }
5
6 type WindowStats struct {
7     // ... существующие поля ...
8
9     // история ключевых метрик по окнам
10    PacketRateHist    *MetricHistory // пакеты в секунду
11    FlowRateHist      *MetricHistory // новые потоки в секунду
12    UniqueSrcHist      *MetricHistory // уникальные srcIP
13    UniqueDstHist      *MetricHistory // уникальные dstIP
14    ClientHelloHist    *MetricHistory // TLS ClientHello в окне
15 }
```

Длина истории (Capacity) — порядка 20–30 окон. Меньше — статистики нестабильны, больше — задержка реакции на смену режима трафика.

2 Статистические примитивы

Файл: internal/stats/primitives.go (создать новый пакет)

Минимальный набор функций, на которые опираются все детекторы. Все принимают срез []float64, возвращают float64 либо (float64, bool) для случаев, когда история слишком коротка.

2.1 Скользящее среднее (SMA) и стандартное отклонение

$$\text{SMA} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \text{SMA})^2}$$

```
1 func Mean(values []float64) (float64, bool)
2 func StdDev(values []float64) (float64, bool) // выборочное;
   делитель N-1
```

Если `len(values) < MinSampleSize` (по умолчанию 5) — возвращать (0, false), детектор должен пропустить статистическую проверку.

2.2 Z-score

$$z = \frac{x_{\text{current}} - \mu_{\text{history}}}{\sigma_{\text{history}}}$$

Текущее значение сравнивается **не само с собой**, а со средним и отклонением по истории *без учёта текущего окна*. Алерт-уровень: $|z| > 3.0$ — сильная аномалия, $|z| > 2.0$ — умеренная.

```
1 func ZScore(current float64, history []float64) (float64, bool)
```

2.3 Перцентили

Используется linear-interpolation алгоритм (как в NumPy `numpy.percentile` с методом `linear`):

```
1 func Percentile(values []float64, p float64) (float64, bool) // p от
   0 до 100
```

Применение: построить p95 и p99 по истории; если `current > p99_history`, метрика в текущем окне — редкий выброс.

2.4 Межквартильный размах (IQR) и MAD

$$\text{IQR} = Q_3 - Q_1, \quad \text{MAD} = \text{median}(|x_i - \text{median}(X)|)$$

```
1 func IQR(values []float64) (float64, bool)
2 func MAD(values []float64) (float64, bool)
```

Использование: точка считается выбросом по правилу Тьюки, если она вне $[Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}]$. Для MAD: робастный аналог Z-score — $z_{\text{mad}} = 0.6745 \cdot (x - \text{median}) / \text{MAD}$, порог $|z_{\text{mad}}| > 3.5$ [4].

2.5 EWMA — экспоненциальное скользящее среднее

$$\text{EWMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EWMA}_{t-1}, \quad \alpha \in (0, 1]$$

```
1 func EWMA(values []float64, alpha float64) (float64, bool)
```

Параметр $\alpha = 0.3$ по умолчанию: даёт быстрый отклик на новые значения, но всё ещё сглаживает шум. Для очень быстрых реакций — $\alpha = 0.5$, для стабильного фона — $\alpha = 0.1$.

2.6 Относительный рост

$$g = \frac{x_{\text{current}}}{\max(x_{\text{previous}}, 1)}$$

Простейший адаптивный сигнал, не требует длинной истории. Используется как первичная проверка резкого изменения.

3 Применение в детекторах

3.1 Перегрузка (Overload)

Файл: internal/detectors/overload_detector.go

К существующим правилам (абсолютный порог pps, рост в K раз — см. отчёт по TLS-метаданным) добавить:

- **Z-score текущего pps относительно истории.** Если $z_{\text{pps}} > 3.0$ при условии $\text{len}(\text{history}) \geq 10$ — сигнал «выброс по сравнению с типичным режимом». Алерт-метаданные дополняются полем ZScore.
- **EWMA-тренд.** Если EWMA пакетов в секунду за последние 5 окон монотонно растёт — пометить как «постепенный рост нагрузки», даже если абсолютный порог ещё не пробит. Это адаптивный сигнал для slowloris-подобных атак.
- **Мягкое снижение основного порога.** Если за последние 3 окна $z > 2.0$ — временно понизить OverloadAbsoluteThreshold на 30%. Возвращать к норме после 5 окон без аномалии.

Обоснование. Голый Z-score плох, потому что трафик не гауссовский — но как один из нескольких сигналов он добавляет ценности. Главное правило (абсолютный порог) остаётся первичным; Z-score только корректирует чувствительность.

3.2 Черви (Worm Detection)

Файл: `internal/detectors/worm_detector.go`

К правилам из TLS-отчёта (доминирующий JA3, число уникальных dstIP) добавить:

- **Перцентильный порог по уникальным dstIP на srcIP.** Вместо фиксированного `WormMinUniqueDst = 20` использовать $\max(20, p95_{\text{history}})$. Это защита от ложных срабатываний в рсар, где у всех хостов высокая активность (например, NAT-шлюз).
- **MAD-выброс по числу TCP SYN от srcIP.** Если у одного srcIP число SYN-пакетов в окне превышает $\text{median} + 3.5 \cdot \text{MAD}/0.6745$ среди всех srcIP того же окна — сильный сигнал сканера. Метод робастен к самим сканерам в выборке [4].
- **Относительный рост уникальных dstIP.** Если у srcIP число уникальных dstIP в текущем окне в ≥ 5 раз больше, чем в предыдущем — пометить как начало пропагации.

Обоснование. Перцентиль адаптирует порог под профиль конкретного рсар, MAD устойчив к ситуации, когда в выборке несколько сканеров одновременно (классическая проблема Z-score: сами выбросы портят baseline).

3.3 DDoS

Файл: `internal/detectors/ddos_detector.go`

К правилам из TLS-отчёта (число потоков на dstIP, доля доминирующего JA3) добавить:

- **Z-score по числу уникальных srcIP на dstIP.** Резкий рост числа уникальных источников на жертву относительно истории — классический сигнал распределённой атаки. Порог $z > 3.0$.
- **p99-фильтр «жертв».** Среди всех dstIP в окне рассматривать как подозрительных только тех, у кого число входящих потоков выше p99 распределения. Это резко уменьшает число false positive на популярных серверах.
- **EWMA сглаживание pps на dstIP.** Использовать EWMA вместо мгновенного pps, чтобы не путать резкий легитимный всплеск (загрузка обновления) с DDoS, развивающимся на протяжении нескольких окон.

Обоснование. DDoS — атака, при которой статистический сигнал особенно силён: распределение источников и пакетов резко отклоняется от нормального профиля жертвы. Z-score и перцентили работают здесь лучше всего из всех детекторов.

3.4 Port scan (если есть в проекте)

Файл: `internal/detectors/portscan_detector.go`

- **IQR-фильтр уникальных `dstPort` на `srcIP`.** Срабатывает, если значение вне $Q_3 + 1.5 \cdot \text{IQR}$ среди всех `srcIP` в окне. Очень точная замена фиксированному порогу.
- **Относительный рост по числу `dstPort`.** Если у `srcIP` в текущем окне $\geq 10\times$ больше уникальных портов, чем в предыдущем — сигнал начала сканирования.

4 Базовые правила и принципы применения

- **Минимальный размер выборки.** Любая статистика требует $N \geq 5$ для слабой оценки, $N \geq 10$ для надёжной. До накопления истории — статистические проверки пропускаются, работают только жёсткие правила.
- **Исключение текущего окна из `baseline`.** При сравнении текущего окна с историей — не включать само текущее окно в подсчёт μ , σ и т.д. Иначе аномалия «сама себя нормализует».
- **Робастные методы — по умолчанию.** Из двух статистик предпочитать ту, что устойчива к выбросам: медиана $>$ среднее, $\text{MAD} > \text{StdDev}$, перцентили $>$ Z-score. Сетевой трафик почти всегда содержит выбросы, и обычные методы их «впитывают» в `baseline`.
- **Статистика не алертит сама по себе.** Срабатывание $z > 3.0$ или $\text{value} > \text{p99}$ — не алерт, а сигнал, который добавляется к контексту существующего детектора или временно повышает его чувствительность.

5 Конфигурация

Параметр	Тип	Значение по умолчанию
<code>HistoryCapacity</code>	int	30 окон
<code>MinSampleSize</code>	int	5
<code>ReliableSampleSize</code>	int	10
<code>ZScoreThresholdStrong</code>	float	3.0
<code>ZScoreThresholdModerate</code>	float	2.0
<code>MADZScoreThreshold</code>	float	3.5
<code>EWMAAlpha</code>	float	0.3
<code>IQRMultiplier</code>	float	1.5
<code>PercentileForVictimFilter</code>	float	99.0
<code>SoftThresholdReduction</code>	float	0.30
<code>SoftThresholdDuration</code>	int	3 окна
<code>SoftThresholdCooldown</code>	int	5 окон

Все параметры выносятся в существующий конфиг (YAML/JSON), новых файлов конфигурации не вводить.

6 Формат вывода

Каждое срабатывание детектора, использующего статистические сигналы, дополняется полем:

```
1 type StatisticalContext struct {
2     Metric      string    // имя метрики; например pps или
   unique_dst
3     Current     float64   // значение в текущем окне
4     BaselineMean float64   // среднее по истории
5     BaselineP95 float64   // p95 по истории
6     ZScore      float64   // z-score текущего значения
7     MADZScore   float64   // робастный z-score через MAD
8     GrowthRatio float64   // current / previous
9     HistorySize int       // длина использованной истории
10 }
```

Это позволит оператору видеть, по какому именно сигналу сработал детектор, и быстро оценивать достоверность.

7 Граничные случаи и ограничения

- **Холодный старт.** Первые `ReliableSampleSize` окон `pcap`-файла — статистики недоступны, детекторы работают только по жёстким правилам. Это допустимая потеря: атаки в первых 10 окнах поймает абсолютный порог.
- **Мультимодальный трафик.** Если `pcap` содержит две разные фазы (например, день и ночь), статистики смешивают их в один `baseline`. В MVP это игнорируется; в будущем — сегментация по фазам через детекцию `change-point` [6].
- **Очень короткие pcap.** Если в файле меньше `HistoryCapacity` окон, статистики не наберут истории; работают только жёсткие правила. Не баг, фича: на коротких файлах статистика ненадёжна.
- **Производительность.** Все статистические функции работают за $O(N)$ или $O(N \log N)$ (для перцентилей через сортировку). $N \approx 30$ окон, вызов на каждое новое окно — порядка десятков микросекунд. Накладные расходы пренебрежимы.
- **Зависимость наблюдений.** Соседние окна сетевого трафика коррелированы — классические оценки σ слегка занижены. Поправок не вводим (это второй порядок малости для нашего применения).

8 Тестирование

К существующим фикстурам (см. отчёт по TLS-метаданным) добавить:

1. **slow_ramp.pcap** — постепенное наращивание rps в течение нескольких минут, ниже абсолютного порога. EWMA-тренд должен поймать; голый порог — нет.
2. **mixed_profile.pcap** — pcap, где первая половина — низкая активность, вторая — высокая (но легитимная). Статистики не должны давать ложных срабатываний.
3. **outlier_noise.pcap** — baseline с несколькими крупными легитимными выбросами (бэкап). MAD/IQR должны корректно отделить их от настоящей аномалии, Z-score — ожидаемо ошибётся.
4. **cold_start_attack.pcap** — атака с первого пакета. Статистика отключена (нет истории), но жёсткие правила должны сработать.

Дополнительно — юнит-тесты на статистические примитивы: проверка корректности на эталонных входах (сравнить с NumPy/SciPy на тех же массивах).

9 Дальнейшие улучшения (вне MVP)

- **CUSUM-фильтр** (cumulative sum) для детекции медленного дрейфа метрики, который не ловится ни абсолютным порогом, ни Z-score [5].
- **Change-point detection** для автоматической сегментации pcap на фазы с разными baseline [6].
- **ARIMA / Holt-Winters** для предсказания нормального значения метрики и оценки отклонения от прогноза. Работает хорошо при наличии периодичности в трафике, что для офлайн-pcap встречается редко.
- **Распределение Пуассона для подсчёта событий**. Число потоков в окне — естественная случайная величина пуассоновского типа; точные доверительные интервалы дают более строгие пороги, чем гауссовские [7].
- **Калибровка порогов на размеченных pcap**. Прогон детекторов на наборе с известными атаками, подбор z -порогов и перцентилей по ROC-кривой.

Список литературы

- [1] NIST/SEMATECH. *e-Handbook of Statistical Methods, Section 6.4: Introduction to Time Series Analysis*. <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc4.htm>

- [2] V. Paxson, S. Floyd. *Wide-Area Traffic: The Failure of Poisson Modeling*. IEEE/ACM Transactions on Networking, vol. 3, no. 3, 1995, pp. 226–244.
- [3] J. W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, 1977.
- [4] C. Leys, C. Ley, O. Klein, P. Bernard, L. Licata. *Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median*. Journal of Experimental Social Psychology, vol. 49, no. 4, 2013, pp. 764–766.
- [5] E. S. Page. *Continuous Inspection Schemes*. Biometrika, vol. 41, no. 1/2, 1954, pp. 100–115.
- [6] S. Aminikhanghahi, D. J. Cook. *A Survey of Methods for Time Series Change Point Detection*. Knowledge and Information Systems, vol. 51, 2017, pp. 339–367.
- [7] J. D. Brutlag. *Aberrant Behavior Detection in Time Series for Network Monitoring*. Proceedings of the 14th USENIX LISA Conference, 2000, pp. 139–146.